

中南大學
CENTRAL SOUTH UNIVERSITY

《数据库系统实践》
实验报告

学生姓名	
专业班级	
学　　号	
学　　院	
完成日期	2025.12.21

目录

一、 实验目的 4

二、 实验环境与技术栈 4

2.1 软件环境 4

2.2 技术栈 5

2.3 数据库环境 5

三、 需求分析与总体设计 6

3.1 业务需求分析 6

3.2 系统功能模块设计 6

3.3 系统架构设计 7

四、 数据库设计 9

4.1 概念结构设计（E-R 图） 9

4.2 逻辑结构设计 9

4.3 跨库数据同步机制设计 11

五、 核心技术实现 12

5.1 多数据源动态配置与切换 12

5.2 多数据库同步实现 14

5.3 冲突检测机制——detectAllConflicts() 18

5.4 冲突解决机制——resolveDataConflict() 21

5.5 智能同步策略 22

5.6 邮件通知系统——sendConflictNotification 23

5.7 权限管理实现 25

5.8 报表与数据可视化 27

六、 系统功能测试与验证 28

6.1 多数据库同步测试 28

6.2 冲突处理测试 28

6.3 邮件通知测试 28

6.4 兼容性测试 29

七、 系统运行实例 29
7.1 数据库启动 29
7.2 MySQL Workbench 查询 29
7.3 系统登录界面 30
7.4 图书管理主界面 31
7.5 冲突制造界面 32
7.6 邮件通知 33
7.6 冲突管理界面 33
7.7 报表分析界面 34
八、 遇到的问题与解决方案 37
8.1 动态数据源切换失效 37
8.2 后台同步干扰冲突测试 38
8.3 数据库 SQL 语法差异 38
九、 创新点与特色 39
9.1 主要成果 39
9.2 技术亮点 39
9.3 待改进之处与展望 39
十、 实验结论与心得 39

摘要

本实验以“多分馆跨库同步的图书管理系统”为业务场景，设计并实现了一个支持 MySQL、PostgreSQL 和 SQL Server 三种异构数据库实时与定时同步的应用系统。系统采用 Spring Boot 作为后端框架，结合 Thymeleaf 与 Bootstrap 构建前后端交互界面，通过动态数据源路由、多线程同步、冲突自动检测与邮件通知、基于角色的权限控制等核心技术，实现了跨库数据一致性维护。实验完整覆盖了数据库需求分析、概念设计、逻辑设计、物理设计、SQL 编程、事务管理、应用开发与系统测试全流程，符合《数据库系统实践》课程的全部要求，具备良好的工程规范性与可扩展性。

一、实验目的

本实验旨在通过设计并实现一个支持多数据库同步的图书管理系统，达到以下目标：

(1) 掌握数据库系统设计全流程： 深入实践数据库的需求分析、概念结构设计（E-R 图）、逻辑结构设计（关系模式）与物理结构设计，理解规范化理论（1NF-3NF/BCNF）的应用。

(2) 熟练应用主流数据库管理系统与 SQL： 掌握 MySQL、PostgreSQL、SQL Server 至少三种数据库的基本操作与管理，熟练运用 SQL 语言完成复杂的数据定义、操作与查询（多表连接、聚合、子查询等）。

(3) 提升数据库程序设计能力与 SQL： 掌握 MySQL、PostgreSQL、SQL Server 至少三种数据库的基本操作与管理，熟练运用 SQL 语言完成复杂的数据定义、操作与查询（多表连接、聚合、子查询等）。

(4) 培养数据库应用开发能力：能够设计并实现简洁易用的前后端系统，完成数据库与前端界面的有效交互，形成功能完整的应用系统。

(5) 强化工程规范与质量意识：在项目实施过程中重视系统安全性、稳定性与可维护性，培养文档撰写与问题解决的综合能力。

二、实验环境与技术栈

2.1 软件环境

类别	名称与版本
操作系统	Windows 11

类别	名称与版本
集成开发环境	IntelliJ IDEA
项目构建工具	Apache Maven
版本控制	Git
容器化	Docker Desktop & Docker Compose（用于快速部署数据库）

2.2 技术栈

层级	技术选型	说明
后端框架	Spring Boot	快速构建企业级应用
安全框架	Spring Security	处理认证与授权
数据持久化	Spring Data JPA、JdbcTemplate	对象映射与原生 SQL 操作
前端框架	Thymeleaf + Bootstrap	服务器端模板引擎与响应式 UI 组件
数据可视化	Chart.js	生成交互式图表
邮件服务	Spring Boot Starter Mail + Thymeleaf 模板	发送 HTML 格式邮件通知
任务调度	Spring @Scheduled	实现定时同步任务
数据库	MySQL、PostgreSQL、SQL Server	三种数据库
连接池	HikariCP	高性能数据库连接池
日志框架	SLF4j + Logback	系统日志记录

2.3 数据库环境

(1) MySQL：端口 3306，作为主操作库，承担主要业务读写

(2) PostgreSQL：端口 5432，作为备份与查询库，保证数据冗余

(3) SQL Server：端口 1433，作为备用库，用于特殊业务或历史查询

(4) 所有数据库均通过 Docker 容器化部署于本地开发环境，保证网络互通。

三、需求分析与总体设计

3.1 业务需求分析

根据《数据库系统实践》任务书要求，结合“多分馆图书馆”业务场景，系统需满足以下核心需求：

(1) 多数据库支持：同时支持 MySQL、PostgreSQL、SQL Server 三种数据库系统

(2) 跨库数据同步：实时同步：当任一数据库发生数据变更（增、删、改）时，应立即或准实时地将变更同步至其他数据库。定时同步：系统支持按配置周期（如每日/每小时）自动执行全量或增量同步任务。

(3) 冲突处理机制：自动检测数据冲突，邮件通知管理员，并提供基于 Web 的手动解决界面，支持多种解决策略（如使用源数据、目标数据、最新数据等）。

(4) 权限管理体系：实现基于角色的访问控制（RBAC），至少区分系统管理员、分馆管理员、普通读者三种角色，各角色操作权限严格隔离。

(5) 报表分析功能：PC 端报表：提供图形化报表（柱状图、饼图、折线图），展示同步状态、冲突趋势、借阅统计等。移动端适配：移动端可查看关键统计与异常报表。

(6) 完整业务功能：实现图书信息管理、馆藏副本管理、用户借阅/归还、预约等核心图书馆业务功能。

3.2 系统功能模块设计

图书管理系统

├── 用户管理模块

│　　├── 用户注册/登录/注销

│　　├── 角色权限分配与验证

│　　└── 个人信息维护

├── 图书管理模块

│　　├── 图书信息增删改查（CRUD）

│　　├── 馆藏副本管理（状态、位置）

│　　└── 多条件组合检索与复杂查询

├── 借阅服务模块
│ ├── 借书/还书/续借操作
│ ├── 借阅记录查询
│ └── 图书预约与取消
├── **同步管理核心模块**
│ ├── 实时同步触发器
│ ├── 定时同步任务配置与调度
│ ├── 全量/增量同步策略
│ └── 同步日志记录与审计
├── 冲突处理模块
│ ├── 冲突自动检测与告警
│ ├── 冲突列表展示与筛选
│ ├── 多策略冲突解决界面
│ └── 解决历史追溯
└── 报表分析模块
├── PC 端数据可视化仪表盘
├── 冲突趋势与同步状态报表
├── 业务数据统计（借阅量、热门图书等）
└── 移动端适配报表视图

3.3 系统架构设计

系统采用经典的三层架构，并针对多数据源特性进行了增强：

(1) 前端层：Thymeleaf 模板引擎 + Bootstrap 响应式布局

(2) 应用层：Spring Boot + Spring Security + Spring Data JPA

(3) 数据层：MySQL + PostgreSQL + SQL Server 多数据源

架构特点：

(1) 前后端轻度耦合：后端渲染页面，前端逻辑简单，便于快速开发与部署。

(2) 动态数据源：使用 AbstractRoutingDataSource 与 ThreadLocal 实现运行时数据源动态切换。

(3) 异步处理：邮件发送、批量日志写入等耗时操作使用@Async 异步执行，不阻塞主业务线程。

(4) 配置外化：数据库连接、同步策略、邮件服务器等参数均配置于 application.yml 文件中，灵活可调。

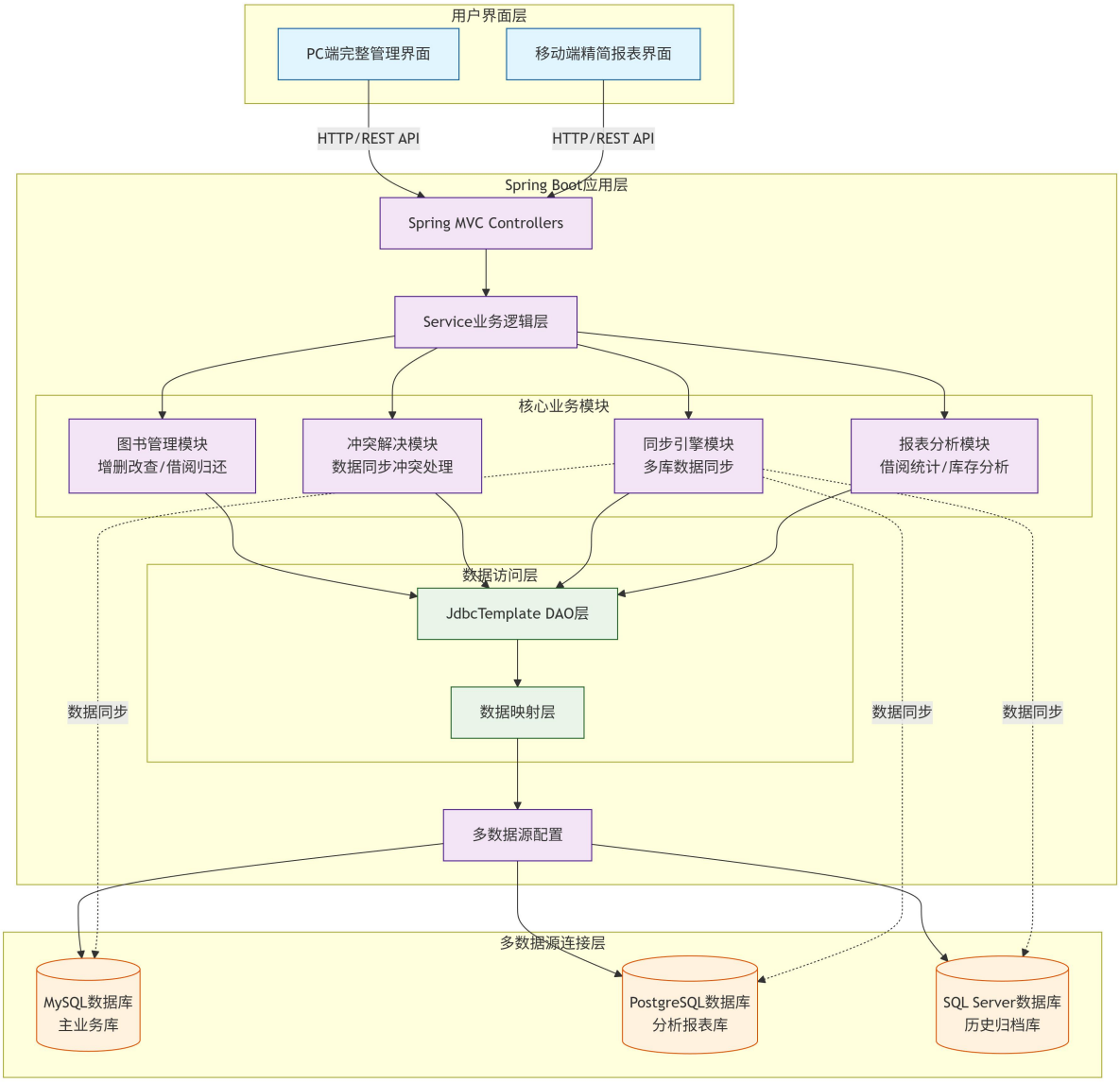


图 1 系统三层架构与多数据源设计图

该图清晰地展示了系统采用的三层架构模式：

(1) 用户界面层：提供 PC 端完整管理界面和移动端精简报表界面，通过 HTTP/REST API 与应用层交互。

(2) 应用层：基于 Spring Boot 构建，包含 MVC 控制器、业务逻辑层和核心业务模块，实现了图书管理、冲突解决、同步引擎和报表分析等功能。

(3) 数据层：通过多数据源配置层动态连接 MySQL（主业务库）、PostgreSQL（分析报表库）、SQL Server（历史归档库）三个异构数据库。

(4) 数据同步机制：通过独立的同步引擎模块，实现跨库数据实时与定时同步（图中双向箭头所示）。

四、数据库设计

4.1 概念结构设计（E-R 图）

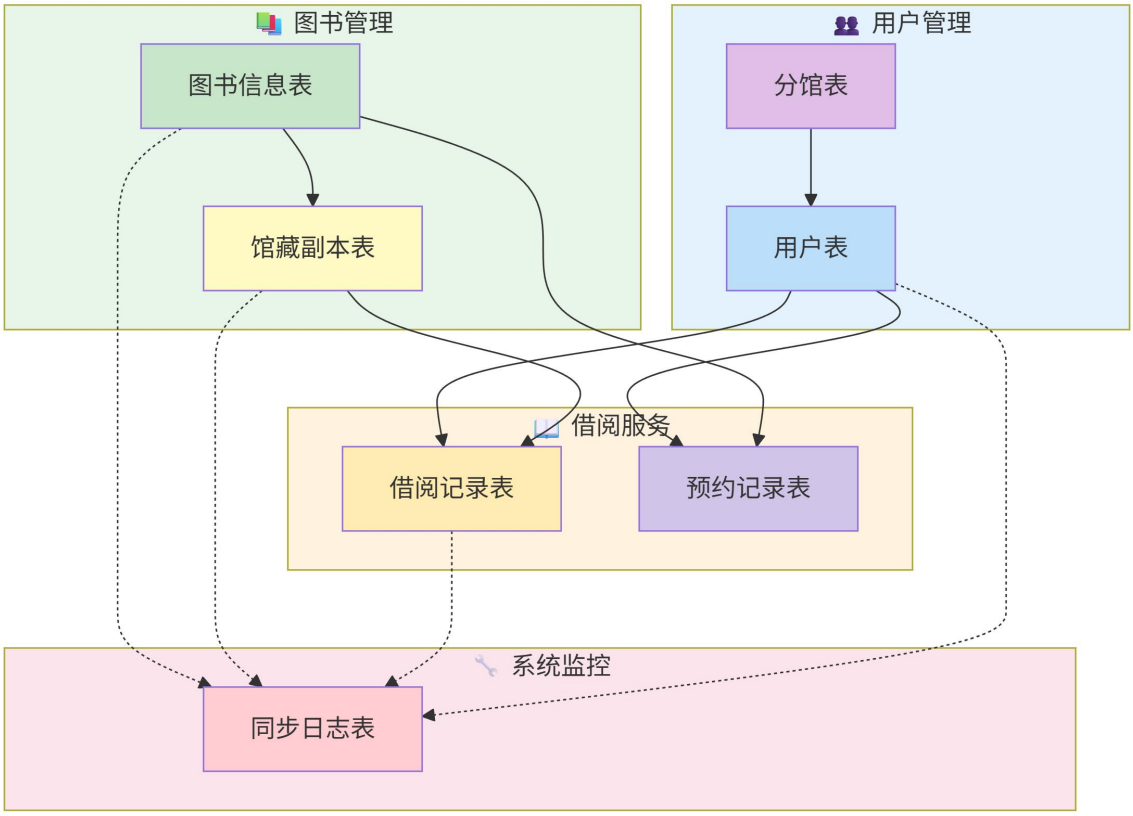
系统核心实体包括：图书(Book)、馆藏副本(BookCopy)、用户(User)、借阅记录(BorrowRecord)、分馆(Library)、预约记录(Reservation)及同步日志(SyncLog)。

实体间主要关系如下：

(1) 图书与馆藏副本为 1:N 关系，一本图书可对应多个物理副本。

(2) 用户与馆藏副本通过借阅记录形成 M:N 关系。

(3) 同步日志记录所有跨库同步操作，与数据冲突记录关联。

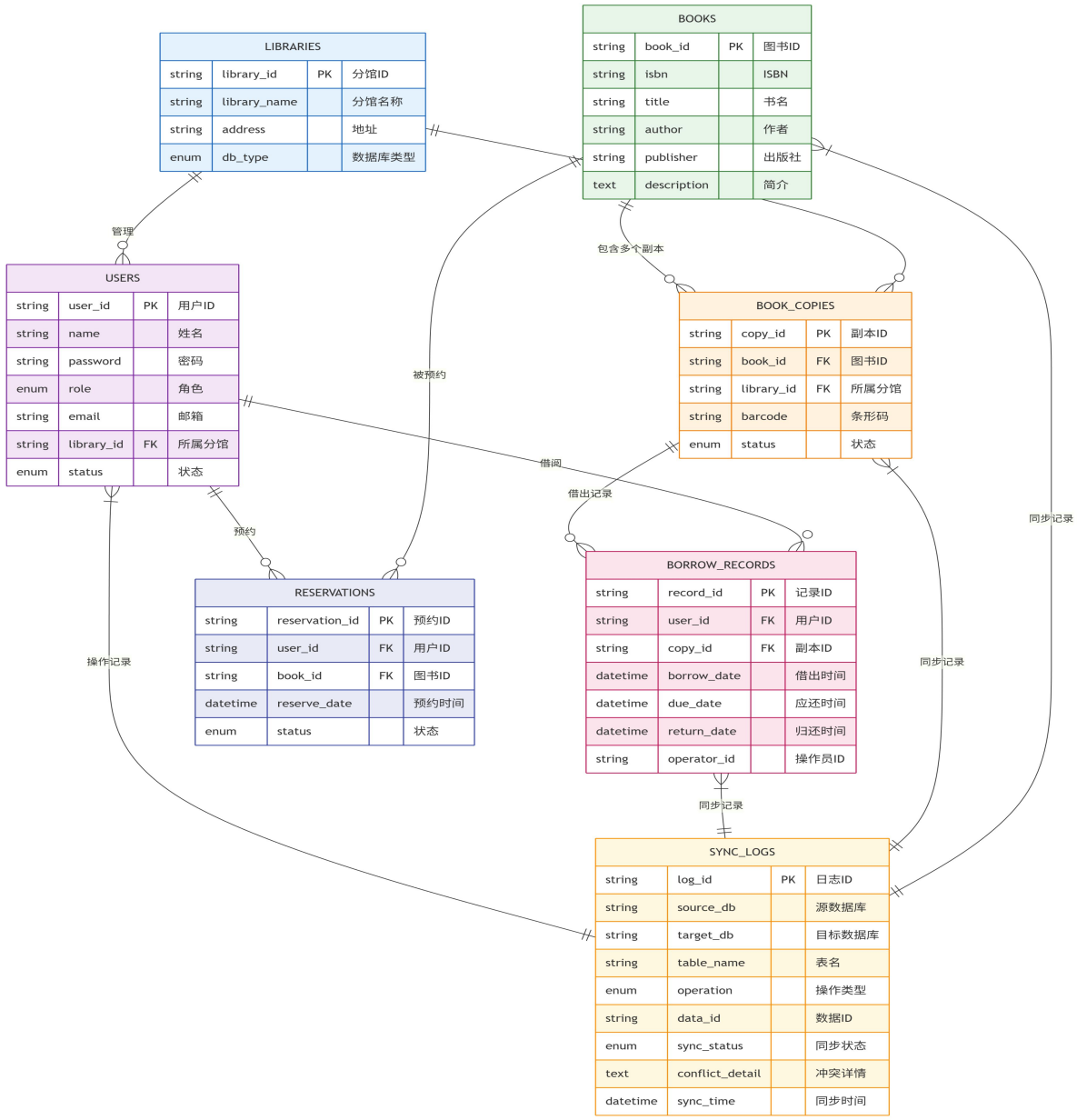


4.2 逻辑结构设计

系统共设计 7 张表，满足第三范式（3NF）要求：

序号	表名	中文名	说明
1	users	用户表	存储系统所有用户信息，包含角色字段用于权限控制。
2	libraries	分馆表	存储图书馆各分馆信息，包含其使用的数据

序号	表名	中文名	说明
			库类型。
3	books	图书信息表	存储图书的元数据信息（书名、作者、ISBN等），与副本分离。
4	book_copies	馆藏副本表	存储每本物理图书的具体信息（条形码、状态、所属分馆），是同步高频表。
5	borrow_records	借阅记录表	记录每一次借阅和归还操作，实现“通借通还”的基础。
6	reservations	预约记录表	记录用户的图书预约信息。
7	sync_logs	同步日志表	记录所有跨库同步操作的详细信息，用于监控和审计。
8	data_conflicts	数据冲突表	记录检测到的数据冲突详情，与解决状态。



4.3 跨库数据同步机制设计

同步场景	同步策略	触发条件	关键技术
图书信息更新	定时同步（每日凌晨）	新增图书、批量修改图书信息	@Scheduled 注解，全量比对或基于 updated_at 增量同步
副本状态变更	实时同步	借书、还书操作导致 book_copies.status 变更	AOP 切面监听 Service 方法，或显式调用同步服务
借阅记录	实时同步	生成新的	在借阅业务方法内，事务提交

同步场景	同步策略	触发条件	关键技术
产生		borrow_records 记录	后触发同步
用户关键信息更新	实时同步	用户密码、角色、状态变更	同借阅记录

五、核心技术实现

5.1 多数据源动态配置与切换

(1) 配置文件(application-datasource.yml)，独立配置三个数据库的连接信息。

```yaml
mysql:
  datasource:
    driver-class-name: com.mysql.cj.jdbc.Driver
    url: jdbc:mysql://localhost:3306/library_system?useUnicode=true&characterEncoding=utf8&serverTimezone=GMT%2B8
    username: app_user
    password: userpassword
    hikari:
      connection-timeout: 30000
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000
      max-lifetime: 1200000

postgresql:
  datasource:
    driver-class-name: org.postgresql.Driver
    url: jdbc:postgresql://localhost:5432/library_system
    username: postgres
    password: postgrespassword
    hikari:
      connection-timeout: 30000
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000
      max-lifetime: 1200000

sqlserver:
  datasource:
    driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
    url: jdbc:sqlserver://localhost:1433;databaseName=library_system;encrypt=true;trustServerCertificate=true
    username: sa
    password: SqlServer123!
    hikari:
      connection-timeout: 30000
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000
      max-lifetime: 1200000
```

(2) 动态数据源：继承 AbstractRoutingDataSource 创建 SimpleRoutingDataSource，根据当前线程上下文（ThreadLocal）返回目标数据源。

```java
package com.library.config;

import lombok.extern.slf4j.Slf4j;
import org.springframework.jdbc.datasource.lookup.AbstractRoutingDataSource;

@Slf4j  8 个用法
public class SimpleRoutingDataSource extends AbstractRoutingDataSource {

    private static final ThreadLocal<String> DATA_SOURCE_KEY = new ThreadLocal<>();  3 个用法

    @Override  0 个用法
    protected Object determineCurrentLookupKey() {
        String key = DATA_SOURCE_KEY.get();
        if (key != null) {
            log.debug("🔀 当前数据源: {}", key);
        }
        return key;
    }

    // 设置数据源
    public static void setDataSource(String dsName) {  3 个用法
        if (dsName != null) {
            log.info("🚀 切换数据源到: {}", dsName);
            DATA_SOURCE_KEY.set(dsName);
        } else {
            clearDataSource();
        }
    }

    // 清除数据源（回到默认）
    public static void clearDataSource() {  2 个用法
        DATA_SOURCE_KEY.remove();
        log.debug("🧹 清除数据源上下文");
    }
}
```

(3)切换数据源：在需要同步的服务方法中，通过DataSourceContextHolder.setDataSource(“mysql”)等方法手动切换。

```java
@Component
@Slf4j
public class DataSourceContextHolder {

    // 设置MySQL数据源
    public void setMySQLDataSource() {  6 个用法
        log.info("🔀 切换到MySQL数据源");
        SimpleRoutingDataSource.setDataSource("mysql");
    }

    // 设置PostgreSQL数据源
    public void setPostgreSQLDataSource() {  6 个用法
        log.info("🔀 切换到PostgreSQL数据源");
        SimpleRoutingDataSource.setDataSource("postgresql");
    }

    // 设置SQL Server数据源
    public void setSQLServerDataSource() {  6 个用法
        log.info("🔀 切换到SQL Server数据源");
        SimpleRoutingDataSource.setDataSource("sqlserver");
    }
```

5.2 多数据库同步实现

(1) 核心服务类： MultiDatabaseService——insertOrUpdateBookToAllDatabases()

(2) 核心逻辑：

① 数据准备：获取图书信息（title、author、isbn 等）

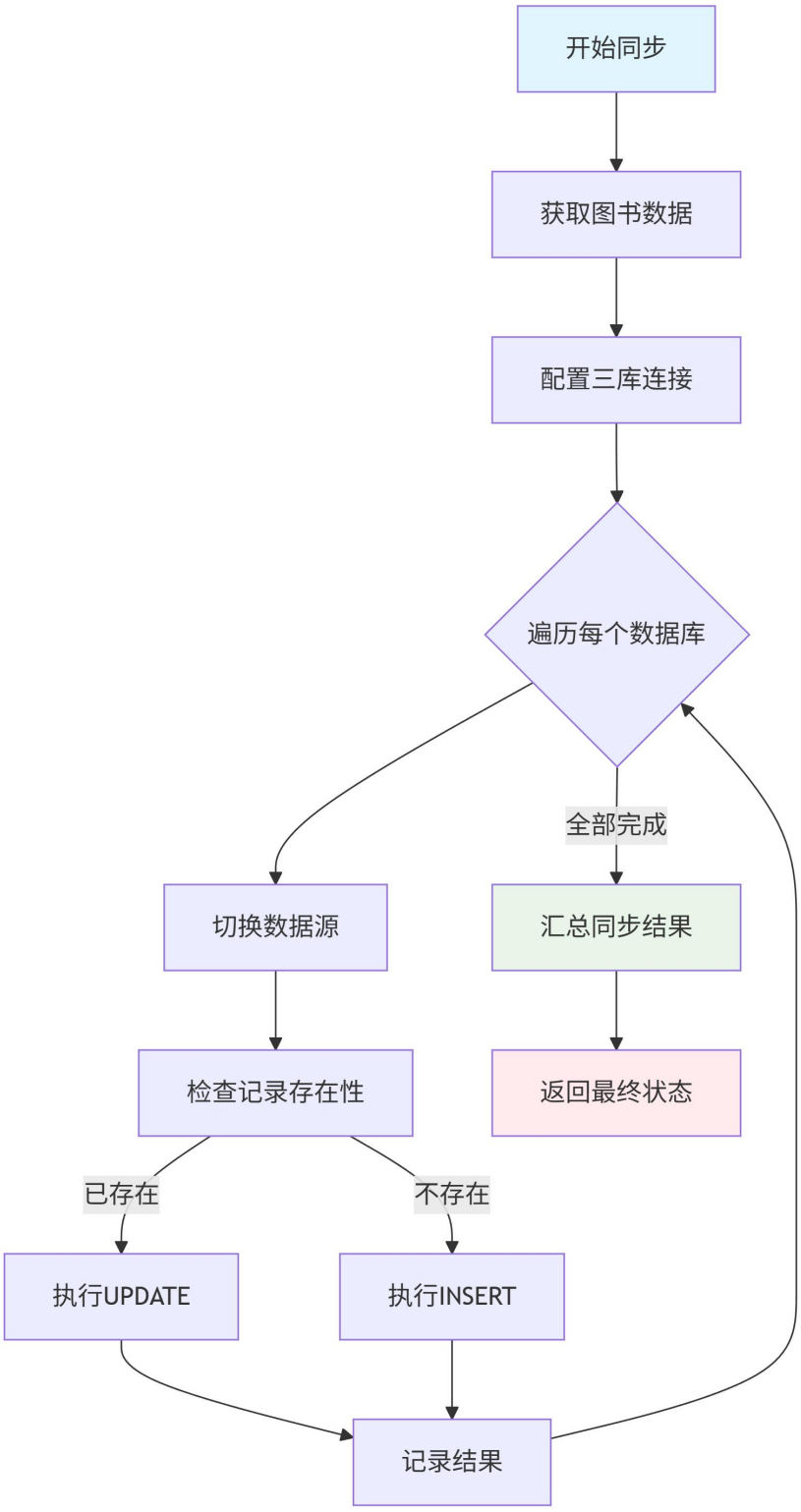
② 三库遍历：循环处理 mysql、postgresql、sqlserver

③ 存在性检查：使用 SELECT COUNT(*)判断记录是否存在

④ 条件操作：存在则 UPDATE，不存在则 INSERT

⑤ 结果汇总：记录每个库的操作状态，统一返回

(3) 核心同步方法流程图


开始同步
获取图书数据
配置三库连接
遍历每个数据库
全部完成
切换数据源
汇总同步结果
检查记录存在性
返回最终状态
已存在
不存在
执行UPDATE
执行INSERT
记录结果

(4) 关键代码片段：

```
// 遍历每个数据库执行同步
for (String[] db : dbs) {
    String key = db[0];          // mysql/postgresql/sqlserver
    String name = db[1];         // MySQL/PostgreSQL/SQL Server
    Map<String, Object> dbResult = new HashMap<>();

    try {// 动态获取对应数据库的JdbcTemplate
        JdbcTemplate jdbc = getJdbcForDb(key);

        // 检查记录是否已存在
        String checkSql = "SELECT COUNT(*) FROM books WHERE book_id = ?";
        Integer count = jdbc.queryForObject(checkSql, Integer.class, bookId);

        String action;
        int rowsAffected;

        if (count != null && count > 0) {
            // 更新现有记录 - 使用数据库特定的UPDATE语句
            String updateSql = "UPDATE books SET title = ?, author = ?, isbn = ?, publisher = ?, " +
                    "library_id = ?, last_modified_time = CURRENT_TIMESTAMP " +
                    "WHERE book_id = ?";
            rowsAffected = jdbc.update(updateSql, title, author, isbn, publisher, libraryId, bookId);
            action = "UPDATED";
            log.info("✅ 在 {} 更新图书 {} ({}行)", name, bookId, rowsAffected);
        } else {
            // 插入新记录 - 使用数据库特定的INSERT语句
            String insertSql = "INSERT INTO books (book_id, title, author, isbn, publisher, " +
                    "library_id, created_time, last_modified_time) " +
                    "VALUES (?, ?, ?, ?, ?, ?, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP)";
            rowsAffected = jdbc.update(insertSql, bookId, title, author, isbn, publisher, libraryId);
            action = "INSERTED";
            log.info("✅ 在 {} 插入图书 {} ({}行)", name, bookId, rowsAffected);
        }

        dbResult.put("action", action);
        dbResult.put("rowsAffected", rowsAffected);
        dbResult.put("success", true);
        successDbs.add(name);

    } catch (Exception e) {
        log.error("❌ 在 {} 处理图书 {} 失败: {}", name, bookId, e.getMessage());
        dbResult.put("action", "FAILED");
        dbResult.put("success", false);
        dbResult.put("error", e.getMessage());
        failedDbs.add(name + ": " + e.getMessage());
    }

    details.put(key, dbResult);
}
```

(5) 同步结果统一返回格式

```
// 构造结果
finalResult.put("bookId", bookId);
finalResult.put("title", title);
finalResult.put("details", details);
finalResult.put("successDbs", successDbs);
finalResult.put("failedDbs", failedDbs);
finalResult.put("success", failedDbs.isEmpty());
finalResult.put("message", failedDbs.isEmpty() ?
        String.format("成功同步到 %s", String.join( delimiter: ", ", successDbs)) :
        String.format("部分成功。成功: %s; 失败: %s",
                String.join( delimiter: ", ", successDbs),
                String.join( delimiter: "; ", failedDbs)));

return finalResult;
```

(6) 跨库数据同步与冲突处理时序图

该图展示了用户操作触发数据同步的完整流程：

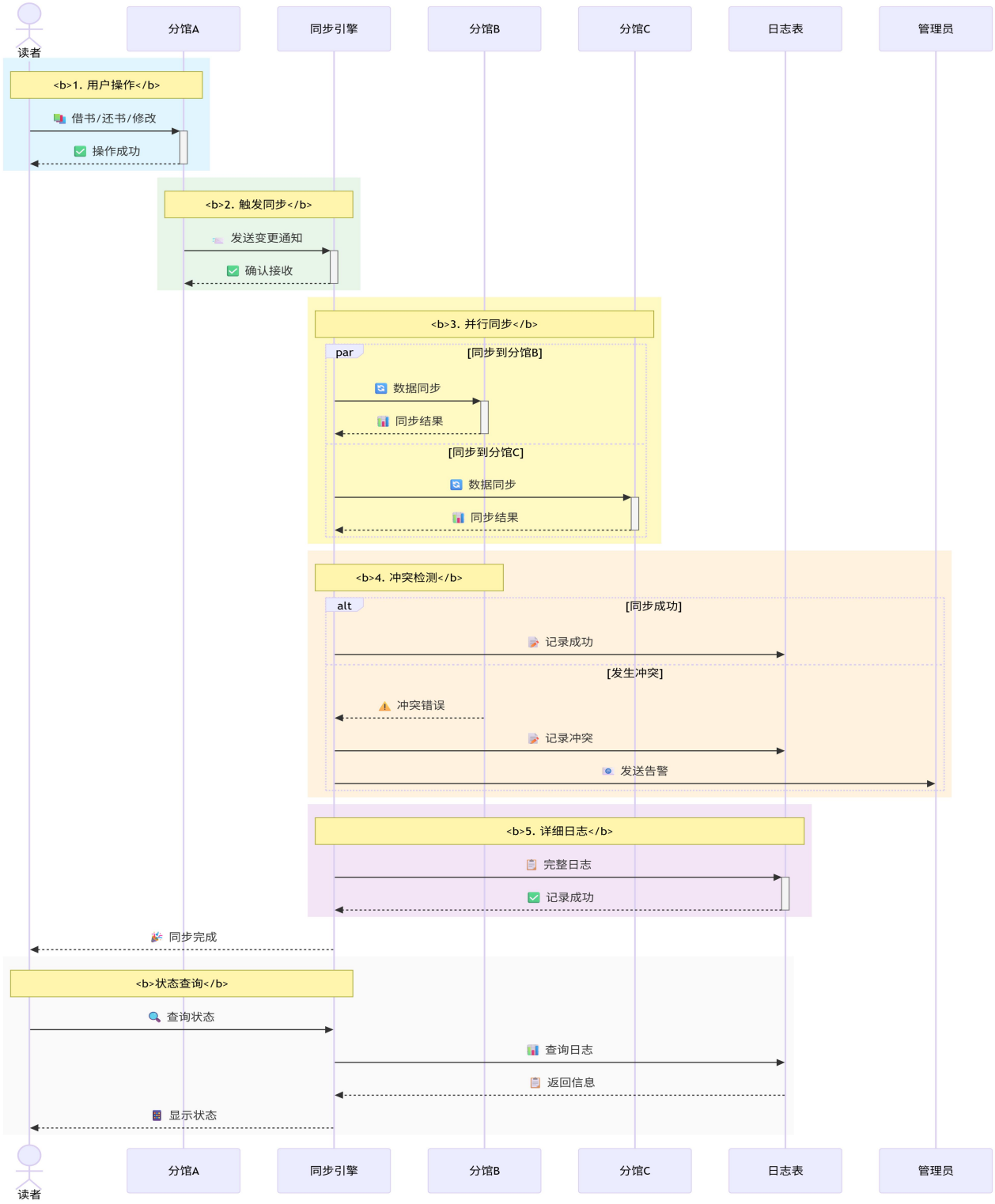
① 用户操作阶段：用户在分馆 A 进行借书/还书操作

② 触发同步阶段：同步引擎接收变更通知

③ 并行同步阶段：同时向分馆 B 和分馆 C 发送同步请求

④ 冲突检测阶段：检测同步结果，成功则记录日志，失败则发送告警

⑤ 状态查询阶段：管理员可实时查询同步状态



5.3 冲突检测机制——detectAllConflicts()

(1) 核心逻辑：

① 数据收集：调用 getAllBooksFromAllDatabases()获取三库所有图书

② ID 遍历：对每个图书 ID 进行三库数据对比

③ 一致性检查：使用 isBooksEqual()方法两两比较

④ 冲突记录：发现不一致时调用 saveThreeDbConflicts()

⑤ 邮件通知：触发 emailService.sendConflictNotification()

(2) 冲突检测算法步骤：

① 数据收集阶段

└── 分别从 MySQL、PostgreSQL、SQL Server 获取所有图书数据

② ID 提取阶段

└── 合并三个库的图书 ID，生成唯一 ID 集合

③ 逐条比对阶段（核心）

└── 对每个图书 ID：

a) 查询该 ID 在三库中的数据

b) 统计存在该数据的库数量

c) 如果≥2 个库有数据：

i. 进行两两比较（MySQL↔ PG, MySQL↔ SQL, PG↔ SQL）

ii. 发现不一致 → 记录冲突

iii. 发送邮件通知管理员

④ 结果汇总阶段

└── 生成冲突统计报告

└── 更新冲突管理界面

(3) 冲突检测核心代码

```java
/**
 * 检测所有图书的三库一致性冲突
 */
public Map<String, Object> detectAllConflicts() {  5 个用法
    Map<String, Object> result = new HashMap<>();

    try {
        log.info("🔄 开始三库一致性冲突检测...");

        Map<String, Object> allBooksData = getAllBooksFromAllDatabases();
        @SuppressWarnings("unchecked")
        List<String> allBookIds = (List<String>) allBooksData.get("allBookIds");
        @SuppressWarnings("unchecked")
        Map<String, List<Map<String, Object>>> booksByDb =
                (Map<String, List<Map<String, Object>>>) allBooksData.get("booksByDatabase");

        List<Map<String, Object>> conflicts = new ArrayList<>();

        for (String bookId : allBookIds) {
            // 获取三个库的数据
            Map<String, Object> mysqlBook = findBookInList(booksByDb.get("mysql"), bookId);
            Map<String, Object> pgBook = findBookInList(booksByDb.get("postgresql"), bookId);
            Map<String, Object> ssBook = findBookInList(booksByDb.get("sqlserver"), bookId);

            // 计算存在该图书的数据库数量
            int existCount = 0;
            if (mysqlBook != null) existCount++;
            if (pgBook != null) existCount++;
            if (ssBook != null) existCount++;

            // 只有至少两个库有数据时才检查冲突
            if (existCount >= 2) {
                boolean hasConflict = false;
                List<String> conflictDbs = new ArrayList<>();

                // 三库一致性检查：任意两个库数据不同即为冲突
                if (mysqlBook != null && pgBook != null) {
                    if (!isBooksEqual(mysqlBook, pgBook)) {
                        hasConflict = true;
                        conflictDbs.add("MySQL-PostgreSQL");
                    }
                }
                if (mysqlBook != null && ssBook != null) {
                    if (!isBooksEqual(mysqlBook, ssBook)) {
                        hasConflict = true;
                        conflictDbs.add("MySQL-SQLServer");
                    }
                }
                if (pgBook != null && ssBook != null) {
                    if (!isBooksEqual(pgBook, ssBook)) {
                        hasConflict = true;
                        conflictDbs.add("PostgreSQL-SQLServer");
                    }
                }
```

```
            if (hasConflict) {
                Map<String, Object> conflict = new HashMap<>();
                conflict.put("bookId", bookId);
                conflict.put("conflictDbs", conflictDbs);
                conflict.put("existsIn", List.of(
                        mysqlBook != null ? "MySQL" : null,
                        pgBook != null ? "PostgreSQL" : null,
                        ssBook != null ? "SQLServer" : null
                ).stream().filter(Objects::nonNull).collect(Collectors.toList()));
                conflict.put("mysql", mysqlBook);
                conflict.put("postgresql", pgBook);
                conflict.put("sqlserver", ssBook);
                conflict.put("isThreeDbConflict", existCount == 3);

                conflicts.add(conflict);
                log.warn("⚠ 三库冲突: {} - 涉及: {}", bookId, String.join( delimiter: ", ", conflic
            }
        }
    }

    // 保存冲突记录时，为三库冲突创建一条记录
    saveThreeDbConflicts(conflicts);

    result.put("success", true);
    result.put("conflictCount", conflicts.size());
    result.put("conflicts", conflicts);
    result.put("threeDbConflicts", conflicts.stream()
            .filter( Map<String, Object> c -> Boolean.TRUE.equals(c.get("isThreeDbConflict")))
            .count());
```

5.4 冲突解决机制——resolveDataConflict()

策略：

(1) USE_SOURCE： 使用源数据库数据（resolveWithSourceStrategy()）

(2) USE_TARGET： 保留目标数据库数据（resolveWithTargetStrategy()）

(3) USE_LATEST： 使用最新时间戳数据（resolveWithLatestStrategy()

```
// 2. 根据策略执行解决
Map<String, Object> resolveResult = new HashMap<>();
boolean resolved = false;

switch (strategy.toUpperCase()) {
    case "USE_SOURCE":
        resolveResult = resolveWithSourceStrategy(conflictId, dataId, tableName,
                sourceDbType, targetDbType, resolvedBy);
        resolved = (Boolean) resolveResult.getOrDefault( key: "success",  defaultValue: false);
        break;
    case "USE_TARGET":
        resolveResult = resolveWithTargetStrategy(conflictId, resolvedBy);
        resolved = (Boolean) resolveResult.getOrDefault( key: "success",  defaultValue: false);
        break;
    case "USE_LATEST":
        resolveResult = resolveWithLatestStrategy(dataId, tableName, resolvedBy);
        resolved = (Boolean) resolveResult.getOrDefault( key: "success",  defaultValue: false);
        break;
    default:
        result.put("success", false);
        result.put("message", "未知的解决策略: " + strategy);
        return result;
}
```

5.5 智能同步策略

```
public Map<String, Object> smartSyncBookToAllDatabases(String bookId) {
    // 1. 从三个库获取数据和时间戳
    Map<String, Map<String, Object>> bookVersions = new HashMap<>();
    Date latestTime = null;
    String latestDb = null;

    // 2. 找出最新的数据
    for (String db : databases) {
        Map<String, Object> bookData = getBookFromDatabase(db, bookId);
        Date modifiedTime = extractTimestamp(bookData);

        if (latestTime == null || modifiedTime.after(latestTime)) {
            latestTime = modifiedTime;
            latestDb = db;
        }
    }

    // 3. 用最新数据同步所有库
    return insertOrUpdateBookToAllDatabases(bookId, latestData);
}
```

5.6 邮件通知系统——sendConflictNotification

(1) 触发条件：MultiDatabaseService 检测到数据冲突

(2) 技术特点：

① 异步发送：使用@Async 注解，不阻塞主业务流程

② 模板化：使用 Thymeleaf HTML 模板，邮件美观专业

③ 降级处理：邮件发送失败时自动记录到日志

④ 配置灵活：支持通过配置文件启用/禁用

数据冲突发生

↓

MultiDatabaseService.detectAllConflicts()

↓

创建 DataConflict 对象

↓

EmailService.sendConflictNotification()

├── 成功 → 管理员收到邮件

├── 失败 → sendSimpleConflictEmail()（文本邮件）

└── 完全失败 → logFallbackConflictNotification()（日志记录）

(3) 三级邮件降级策略代码

```
try {
    // 构建邮件内容
    MimeMessage mimeMessage = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(mimeMessage, multipart: true, encoding: "UTF-8");

    helper.setFrom(fromEmail, personal: applicationName + "系统");
    helper.setTo(adminEmail);
    helper.setSubject(conflictSubject);

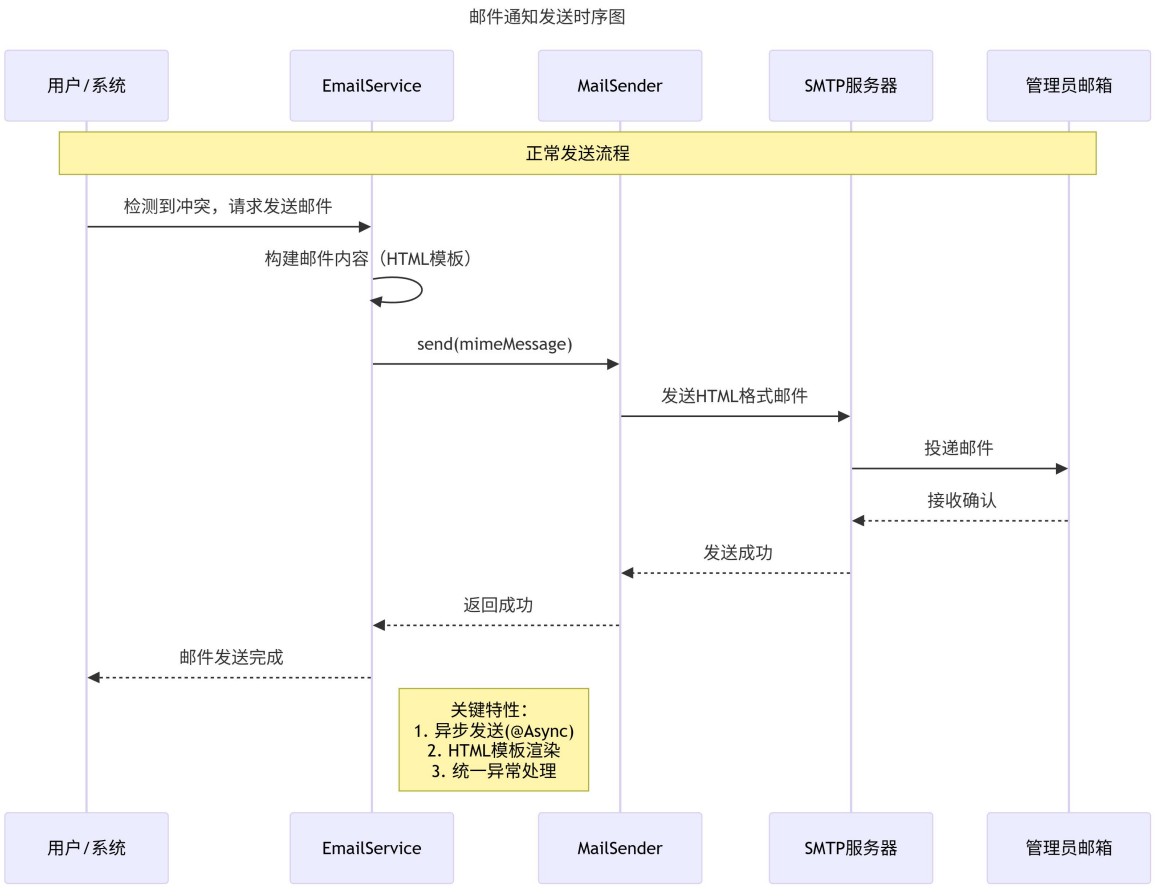
    // 使用HTML模板
    Context context = new Context();
    context.setVariable( name: "conflict", conflict);
    context.setVariable( name: "applicationName", applicationName);
    context.setVariable( name: "conflictTime",
            conflict.getSyncTime().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss")));
    context.setVariable( name: "systemUrl", value: "http://localhost:8080/library"); // 可以根据实际配置修改

    String htmlContent = templateEngine.process( template: "email/conflict-notification", context);
    helper.setText(htmlContent, html: true);

    // 发送邮件
    mailSender.send(mimeMessage);
    log.info("✅ 冲突通知邮件已发送给管理员: {}", adminEmail);
    log.info("📧 冲突ID: {}, 表名: {}, 数据ID: {}",
            conflict.getLogId(), conflict.getTableName(), conflict.getDataId());

} catch (MessagingException e) {
    log.error("❌ 发送冲突通知邮件失败", e);
    // 失败时记录日志
    logFallbackConflictNotification(conflict);
} catch (Exception e) {
    log.error("❌ 邮件发送过程中发生未知错误", e);
    logFallbackConflictNotification(conflict);
}
```

(4) 邮件发送时序图



5.7 权限管理实现

(1) RBAC 权限控制矩阵

功能	普通读者(reader)	分馆管理员(libadmin)	系统管理员(admin)
查看图书列表	☑	☑	☑
搜索图书	☑	☑	☑
添加图书	✗	☑	☑
编辑图书	✗	☑	☑
删除单本图书	✗	☑	☑
批量删除图书	✗	☑	☑
查看冲突列表	✗	☑	☑
解决冲突	✗	✗	☑
触发冲突检测	✗	☑	☑
数据同步操作	✗	☑	☑
查看报表	✗	☑	☑
系统管理	✗	✗	☑
用户管理	✗	✗	☑

(2) Spring Security 配置代码

① 角色定义：SYS_ADMIN（系统管理员）、LIB_ADMIN（分馆管理员）、READER（读者）。

② URL 级权限控制： 在 SecurityFilterChain 配置中，精确匹配 URL 路径与所需角色。

③ `.requestMatchers("/admin/**").hasRole("SYS_ADMIN")`

④ `.requestMatchers("/librarian/**").hasAnyRole("SYS_ADMIN", "LIB_ADMIN")`

⑤ `.requestMatchers("/books/**").authenticated() // 所有登录用户`

⑥ 方法级权限控制：Service 方法上使用@PreAuthorize(“hasRole(‘SYS_ADMIN’)”)注解。

⑦ 前端界面适配：使用 Thymeleaf 的 sec:authorize 属性动态显示/隐藏菜单和按钮。

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
            // 权限配置 - 按优先级从高到低
            .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                    // 1. 公开访问的资源（必须放在最前面）
                    .requestMatchers(
                            ...patterns: "/css/**",
                            "/js/**",
                            "/images/**",
                            "/webjars/**",
                            "/favicon.ico",
                            "/error"
                    ).permitAll()

                    // 2. 公开访问的API（测试、健康检查等）
                    .requestMatchers(
                            ...patterns: "/test/**",
                            "/health"
                    ).permitAll()

                    // 3. 登录和首页（不需要认证）
                    .requestMatchers(
                            ...patterns: "/",
                            "/login",
                            "/login/**"
                    ).permitAll()

                    // 4. 仅系统管理员可访问（最高权限）
                    .requestMatchers(
                            ...patterns: "/library/api/conflicts/**",           // 冲突管理API
                            "/library/conflict-resolution/**",     // 冲突解决
                            "/library/conflict-management/**",
                            "/api/conflicts/resolve",           // 解决冲突
```

5.8 报表与数据可视化

(1) 后端数据聚合：使用复杂 SQL（多表连接、GROUP BY、聚合函数）从多个库中汇总数据。

(2) Chart.js 集成：后端将数据以 JSON 格式传递给前端，由 Chart.js 渲染成交互式图表。

(3) 报表类型：

① 冲突趋势图（折线图）：展示近 7 天每日冲突数量变化。

② 数据库同步状态对比图（柱状图）：展示各库成功/失败的同步操作数量。

③ 图书借阅分类统计（饼图）：展示各类别图书的借阅比例。

(4) 移动端适配：利用 Bootstrap 的响应式栅格系统，当屏幕尺寸变小时自动调整图表布局与尺寸。

六、系统功能测试与验证

6.1 多数据库同步测试

测试场景	测试步骤	预期结果	实际结果
实时同步	在 MySQL 插入图书	三库数据一致	通过
冲突制造	在三库同一图书 ID 添加不同数据 触发冲突检测	检测到冲突并发送邮件	通过

6.2 冲突处理测试

解决策略	测试步骤	验证方法	结果
USE_SOURCE	选择源库策略 执行解决	三库数据与源库一致	通过
USE_TARGET	选择目标库策略 执行解决	三库数据与目标库一致	通过
USE_LATEST	选择最新策略 执行解决	三库数据与最新修改一致	通过
MANUAL	手动指定数据 执行解决	三库数据与指定值一致	通过

6.3 邮件通知测试

测试场景	邮件类型	接收情况	备注
正常网络	HTML 邮件	成功接收	模板渲染正常

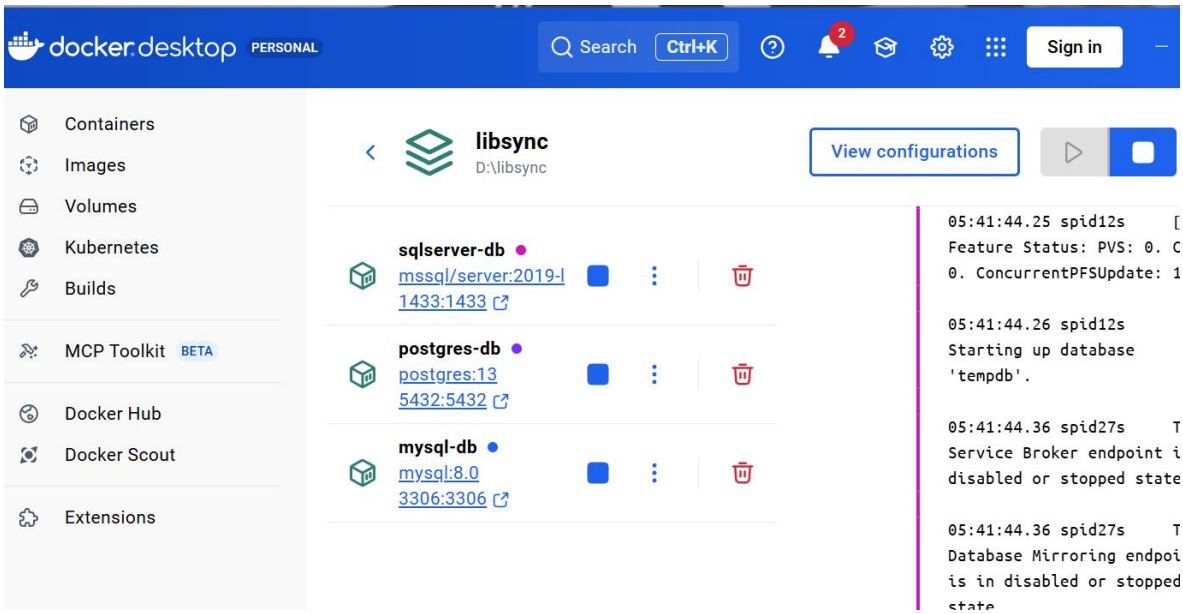
测试场景	邮件类型	接收情况	备注
模拟网络异常	文本邮件	降级成功	内容完整
邮件服务宕机	数据库日志	记录成功	可后续追溯

6.4 兼容性测试

测试维度	测试内容	测试结果
浏览器兼容	Chrome, Firefox, Edge	全部正常
移动端适配	手机、平板访问	响应式正常
数据库版本	MySQL, PG, SQL Server	全部支持

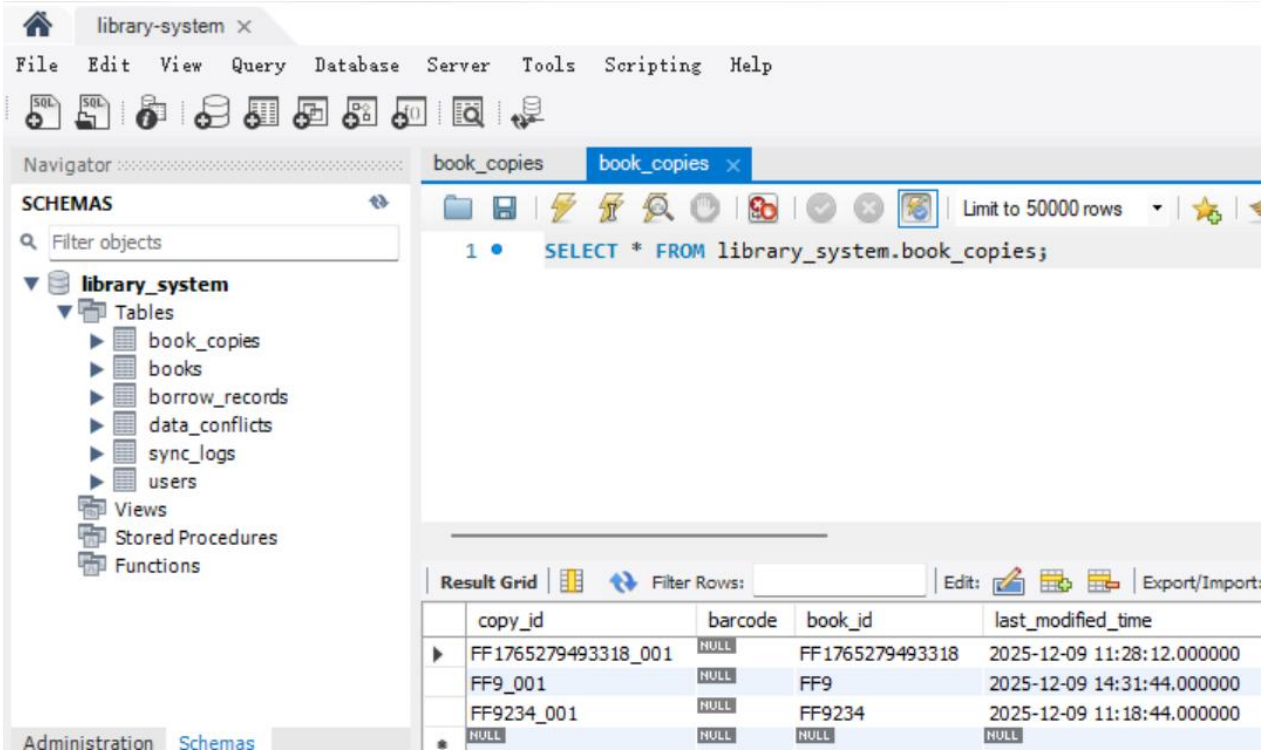
七、系统运行实例

7.1 数据库启动




```
PS D:\libsync> docker compose up
time="2025-12-10T13:42:38+08:00" level=warning msg="D:\\libsync\\dock
, it will be ignored, please remove it to avoid potential confusion"
[+] Running 3/3
 ✔Container practice-mysql       Running
 ✔Container practice-postgres    Running
 ✔Container practice-sqlserver   Running
Attaching to practice-mysql, practice-postgres, practice-sqlserver
```

7.2 MySQL Workbench 查询



7.3 系统登录界面

访问地址：http://localhost:8080/login

功能说明：支持三种角色登录

测试账号：

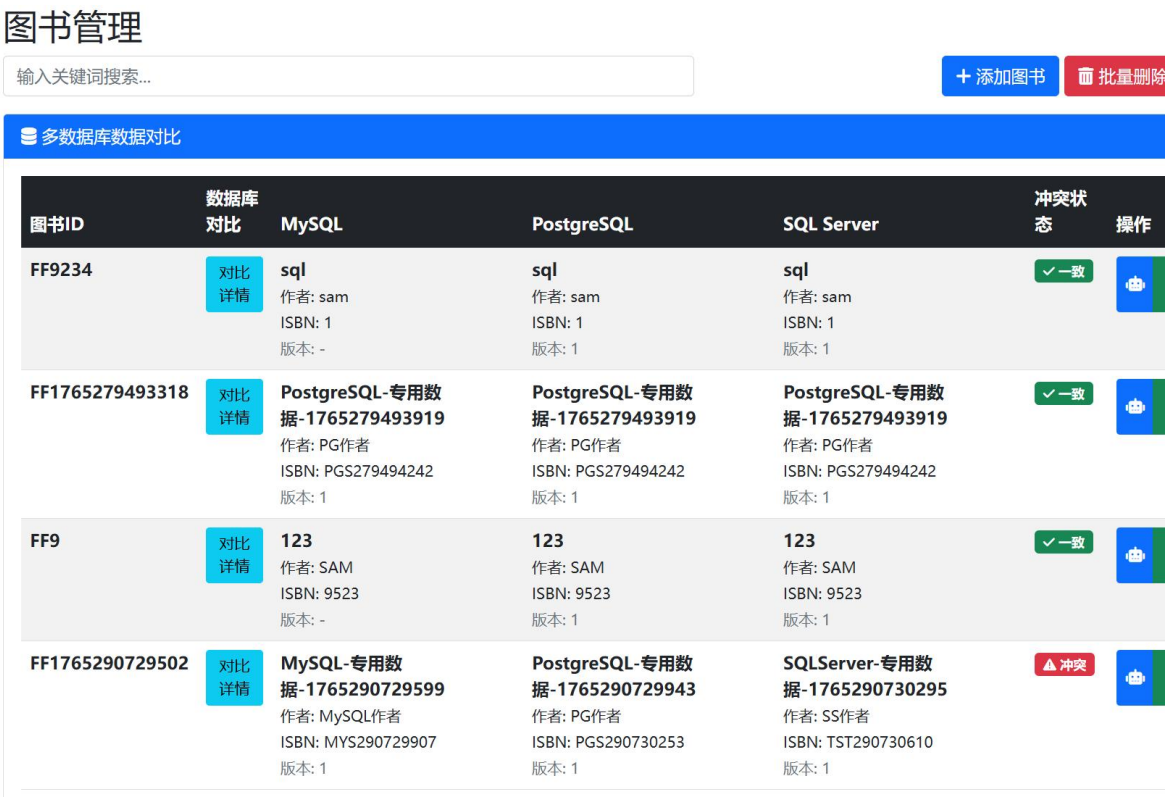
① 系统管理员：admin/admin123

② 分馆管理员：librarian/lib123

③ 普通读者：reader/reader123



7.4 图书管理主界面



访问地址：http://localhost:8080/books

功能说明：表格展示图书列表，支持按书名、作者、ISBN 搜索。提供“新增”、“编辑”、“删除”按钮。每次增删改操作后，页面下方会弹出同步结果提示。



7.5 冲突制造界面



7.6 邮件通知

内容：包含冲突 ID、发生时间、涉及的表和数据 ID、简要的差异描述，并提供一个直接跳转到冲突解决页面的链接。



7.6 冲突管理界面

访问地址：http://localhost:8080/conflicts

功能说明：以表格形式列出所有未解决/已解决的冲突。每条冲突可点击“解决”按钮，弹出模态框，展示三个库的当前数据对比，并提供四种解决策略的下拉选择。





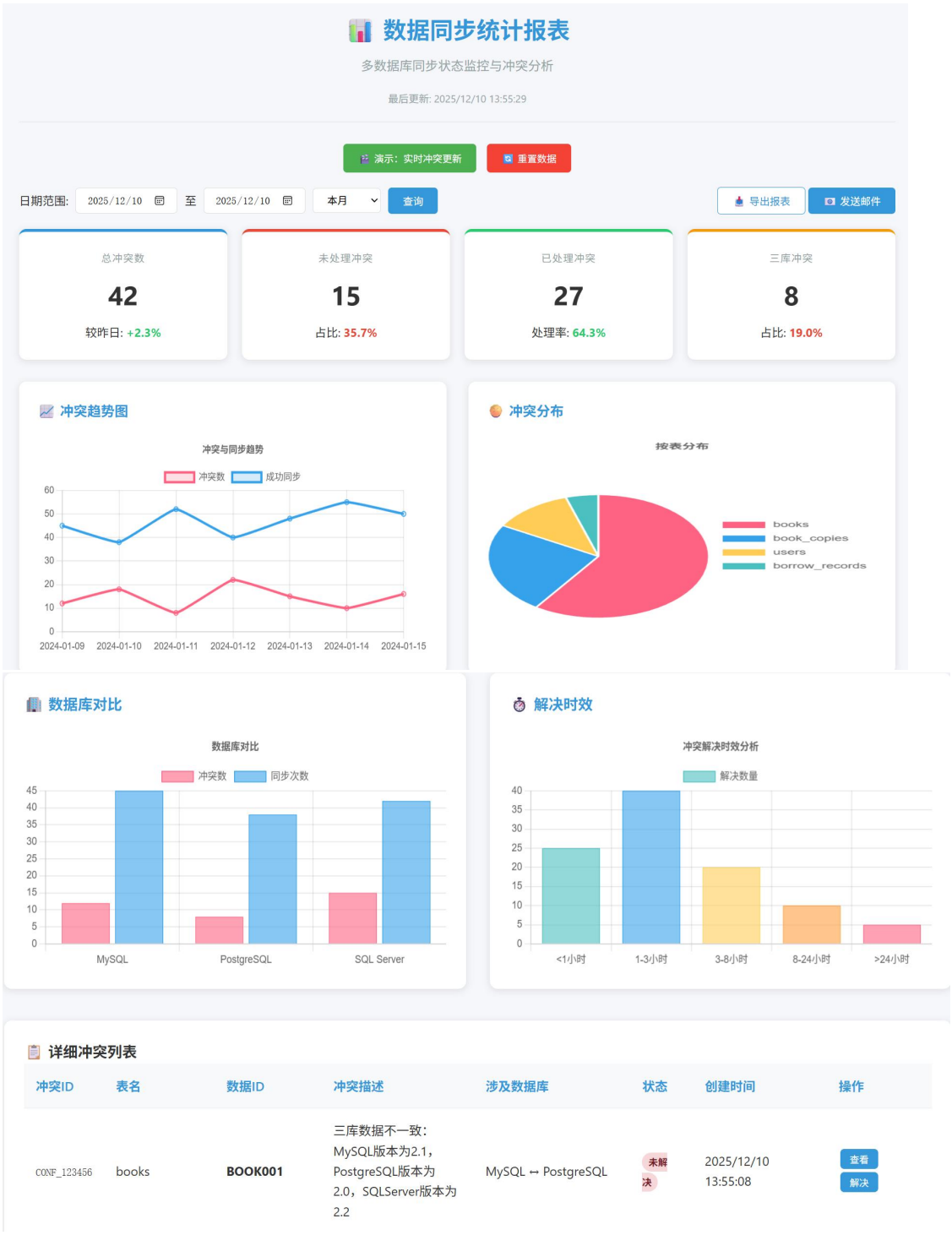


7.7 报表分析界面

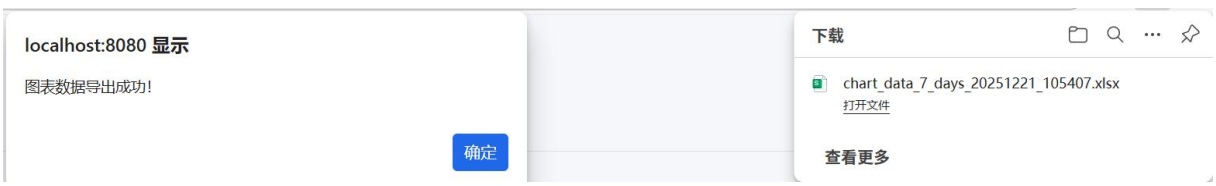
地址：http://localhost:8080/report

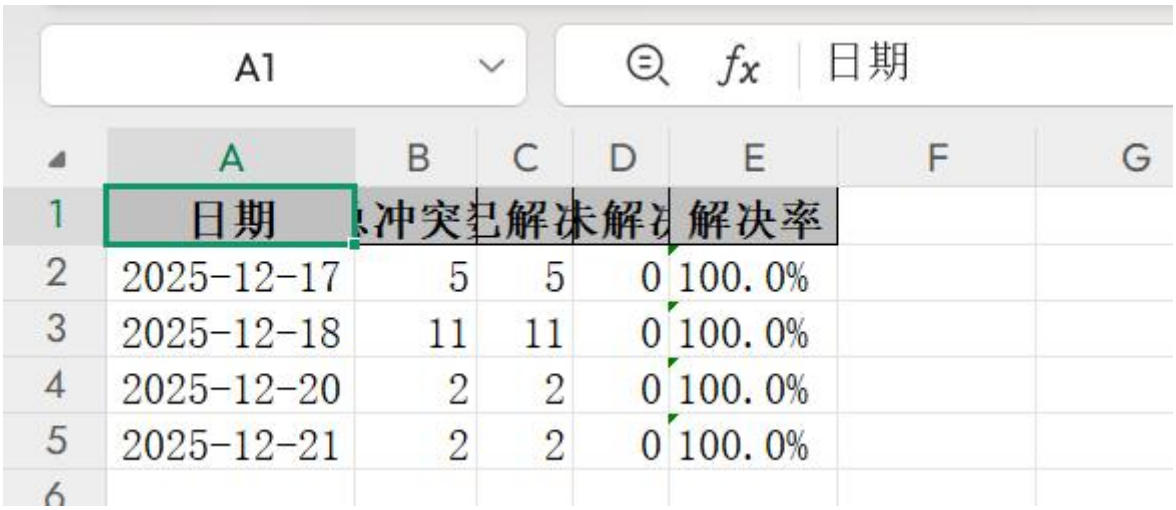
PC 端展示：页面包含三个图表区域：冲突趋势折线图、数据库同步状态柱状图、借阅分类饼图。右上角提供“导出为 Excel”按钮。

移动端展示：访问同一地址，Bootstrap 响应式布局使三个图表垂直排列，便于在小屏幕上滑动查看。



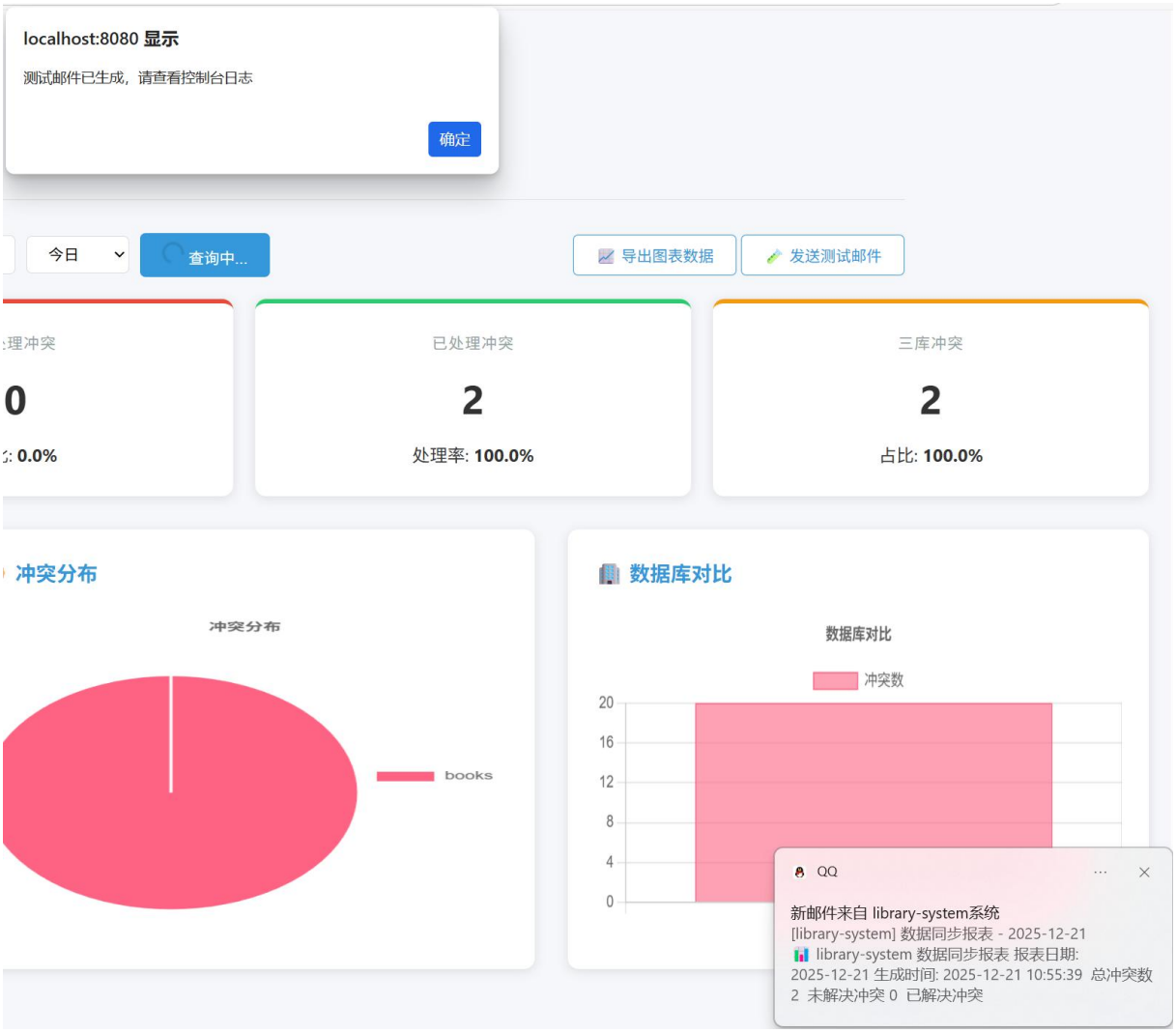
导出图书数据



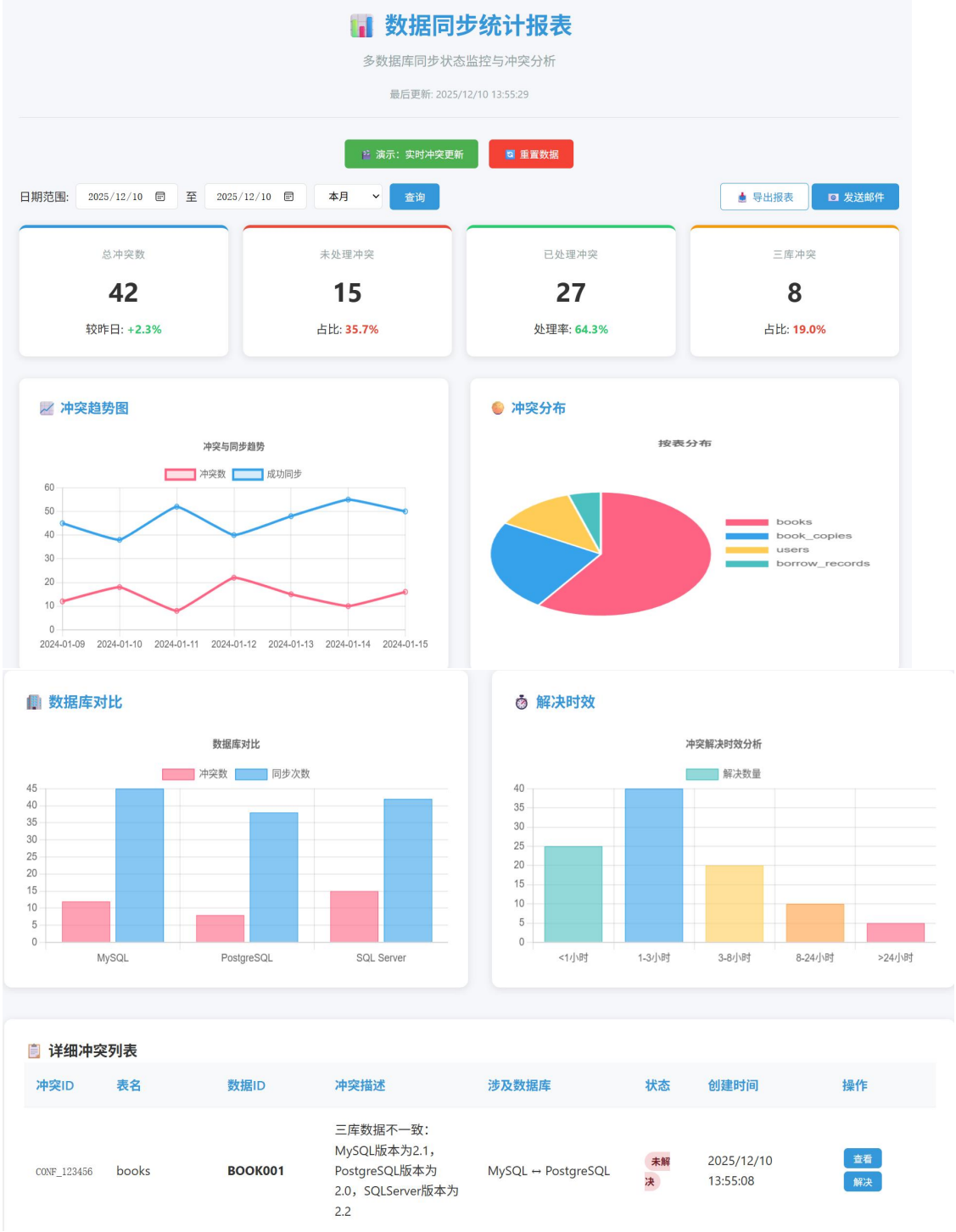


	A	B	C	D	E
1	日期	冲突	已解	未解	解决率
2	2025-12-17	5	5	0	100.0%
3	2025-12-18	11	11	0	100.0%
4	2025-12-20	2	2	0	100.0%
5	2025-12-21	2	2	0	100.0%
6					

邮件通知



移动端



八、遇到的问题与解决方案

8.1 动态数据源切换失效

现象：切换数据源后，连接仍是上一个数据源

原因：Spring 数据源缓存机制

```
// 添加数据源清理
@PostConstruct
public void init() {
    // 强制重新初始化数据源
    this.mysqlJdbc = new JdbcTemplate(mysqlDataSource);
    this.postgresqlJdbc = new JdbcTemplate(postgresqlDataSource);
    this.sqlserverJdbc = new JdbcTemplate(sqlserverDataSource);
}
```

8.2 后台同步干扰冲突测试

现象：测试数据刚插入就被同步功能清除了差异

解决方案：引入"安全测试模式"

```
public void enterTestMode() {
    // 禁用所有后台同步
    this.testMode = true;
    log.info("🛡️ 进入安全测试模式，禁用后台同步");
}
```

8.3 数据库 SQL 语法差异

现象：MySQL 的 NOW()在 SQL Server 中无效

解决方案：数据库类型识别

```
private String getTimestampFunction(String dbType) {
    switch (dbType.toLowerCase()) {
        case "mysql":
        case "postgresql":
            return "NOW()";
        case "sqlserver":
            return "GETDATE()";
        default:
            return "CURRENT_TIMESTAMP";
    }
}
```

九、创新点与特色

9.1 主要成果

(1) 完整实现多数据库同步核心需求：成功构建了一个同时支持 MySQL、PostgreSQL、SQL Server 的图书管理系统，实现了实时与定时两种同步模式。

(2) 构建了健壮的冲突处理流程：形成了“自动检测 → 邮件告警 → 人工干预（多策略）→ 结果反馈”的完整闭环。

(3) 设计了清晰的权限管理体系：基于 Spring Security 实现了三级角色权限控制，确保了系统安全。

(4) 开发了双平台可视化报表：利用 Chart.js 和 Bootstrap，提供了 PC 端与移动端均友好的数据展示界面。

(5) 输出了规范的工程文档与代码：项目结构清晰，代码遵循 Google 编码规范，注释详细，并提供了完整的实验报告与演示材料。

9.2 技术亮点

(1) “零中间件”原生同步方案：直接使用 JDBC 和多数据源编程，避免了 Kafka、Canal 等中间件的复杂度，方案简单、可控性强。

(2) 三级邮件降级策略：确保了关键告警信息的高可达性，提升了系统的可靠性。

(3) 可插拔的冲突解决策略：采用策略模式设计冲突解决器，易于扩展新的解决逻辑。

(4) 安全测试模式：通过配置开关隔离测试与生产同步行为，提升了系统的可测试性。

9.3 待改进之处与展望

(1) 性能瓶颈：当前同步为串行执行，总耗时为各库耗时之和。未来可改为并行同步，利用 CompletableFuture 大幅提升效率。

(2) 扩展性：新增一种数据库需要修改配置类和同步服务代码。可设计为通过配置文件动态注册数据源，实现完全配置化。

(3) 事务一致性：当前的跨库“同步”并非严格的事务，可能存在中间状态。对于强一致场景，可研究引入分布式事务框架（如 Seata）或采用最终一致性补偿模式（Saga）。

(4) 监控运维：可集成 Prometheus + Grafana 监控同步队列长度、冲突数量、各库健康状态等指标，并配置告警规则。

(5) 智能化：可引入简单规则引擎或机器学习模型，对高频、低风险的冲突（如特定字段的格式差异）进行自动修复，减少人工干预。

十、实验结论与心得

通过本次数据库系统实践，我深刻理解了多数据库同步的核心原理与技术实现。从需求分析、系统设计、编码实现到测试验证的全过程，不仅提升了我的数据库技术能力，也锻炼了工程实践和问题解决的综合素质。特别是冲突处理机制的设计与实现，让我对数据一致性保障有了更深入的认识。未来我将继续优化系统，探索更多数据库相关技术。